LLD Phase 5 — PDF 6

Amazon Locker · Notification System · Inventory Management · Ride-sharing | Requirements first, hints in the middle, full solutions at the back

> **Colour key. Purple = the interviewer's words · Blue = a note or hint · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

Contents

Before you start — the loop · the rush helper · time boxes · warm-up: two PDF 5 designs from memory, then 12 quick ones

Problem 1 — Amazon Locker (small)

Problem 2 — Notification System (core)

Problem 3 — Inventory Management (core)

Problem 4 — Ride-sharing (heavy)

Hints — three levels per problem: a nudge, the shape, the key decision

Solutions 1–4 — clarifying questions · Part A model answer · what to write first · full code · Part C answers · common mistakes · 10-point checklist

Back pages — warm-up answers · mistake log · re-do schedule · all 24 problems on one page · what to do next

Before you start

This is the last PDF of Phase 5, in the same format as PDFs 3 to 5: the interviewer's words in purple first (write your own questions before reading on), interview-sized solutions, and a "write this first" order in each. The loop stays the same: Part A on paper, one hint level only after 15 minutes stuck, Part B until the output matches exactly, Part C out loud, then the solution and its checklist.

New in PDF 6	What it means for you
Doing it once, safely	Callers retry. The notification service must send a request once even when it arrives twice at the same moment. This is *idempotency*, asked about in almost every backend interview.
Promised vs on the shelf	Inventory keeps "on hand" and "reserved" apart, so a checkout that never pays can be undone cleanly. The same idea is behind seat holds and hotel rooms.
Claiming without a big lock	Ride-sharing claims a driver with `compareAndSet`: one atomic step on the driver, no lock over the whole city. Compare it with Food Delivery's single lock.

The rush helper (given, same as before)

```
// Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
static int rush(int n, IntPredicate task) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(n);
    CountDownLatch go = new CountDownLatch(1);
    List<Future<Boolean>> results = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int id = i;
        results.add(pool.submit(() -> { go.await(); return task.test(id); }));
    }
    go.countDown();
    int trues = 0;
    for (Future<Boolean> f : results) if (f.get()) trues++;
    pool.shutdown();
    return trues;
}
```

Used by the Notification System, Inventory Management and Ride-sharing. It needs `import java.util.concurrent.*;` and `import java.util.function.IntPredicate;`, and `main` must declare `throws Exception`.

Problem	Size	Time box	Format
1. Amazon Locker	small	45 min	10 design + 35 code
2. Notification System	core	75 min	20 design + 55 code
3. Inventory Management	core	75 min	20 design + 55 code
4. Ride-sharing	heavy	100 min	25 design + 75 code

Warm-up (15 minutes)

Part 1 — from memory. Don't open PDF 5. Write on paper, then check against the back pages.

W1	**Task Scheduler** (5 min). Which two structures hold "not due yet" and "due, waiting for a worker", and in what order is each kept? Write the three steps of one second, in order. Fixed rate or fixed delay, and why? What does cancel do to a running task?
W2	**Food Delivery** (5 min). Draw the order's life cycle. Why an enum with allowed moves instead of the State pattern? What must happen in one step when assigning a partner? What happens when a partner is freed?

Part 2 — name the pattern (5 min). All from one app: a housing society app like MyGate, for a society in Baner. Name the pattern, or say "nothing" and what you'd use instead.

1. Maintenance charges: per square foot for flats, a fixed amount for shops, a discount for paying early. The committee keeps changing the rules.
2. A visitor entry goes EXPECTED → AT_GATE → APPROVED or DENIED → INSIDE → EXITED. Each step just checks it's allowed, changes the status and logs it.
3. The society's old intercom offers `ring(int flatCode)`. Our code calls `Intercom.call(Flat)` everywhere.
4. When a parcel arrives at the gate: notify the resident, add it to the gate log, update the guard's screen. More reactions will come.
5. Purchases: the facility manager approves up to Rs 5,000, the treasurer up to Rs 50,000, above that the committee.
6. A gate pass PDF can get a society logo, a QR code and a watermark, in any combination.
7. The society has towers, towers have floors, floors have flats. Unpaid dues are needed at every level.
8. A complaint not resolved in 2 days moves to the secretary.
9. Monthly reports (dues, visitors, parking) all go: fetch data, work out totals, format, email. Only fetching and totals differ.
10. The payment gateway is chosen by a name in the config: "razorpay", "paytm" or "phonepe".
11. Every resident can read the notice board, only committee members can post, and the list is kept for 5 minutes to save database calls.
12. A flat has a tower, a number and an area, which never change.

Problem 1 — Amazon Locker

Small · 45 minutes (10 design + 35 code) · machine-coding format

> **The interviewer says:** "Design an Amazon Locker. A locker station near your society has compartments of different sizes. The delivery agent drops a parcel, the customer gets a code, and picks the parcel up by entering the code. Parcels that aren't collected in time go back to the seller."

After your questions, the requirements are

R1	Compartments have unique IDs and a size: SMALL, MEDIUM or LARGE.
R2	Dropping a parcel (an ID, a customer, a size) puts it in the smallest free compartment it fits in: a SMALL parcel can go in a MEDIUM compartment if no SMALL one is free. Within a size, the compartment added first. Refused if the parcel ID was used before, or nothing fits.
R3	Each parcel gets a 6-digit pickup code, unique among the parcels in the locker. The locker is given its code generator; the test uses one that returns 100001, 100002, ...
R4	The right code opens the compartment and frees it. A wrong code is refused. After 3 wrong codes in a row, every code is refused until the next day; a right code resets the count.
R5	Days start at 1. A parcel dropped on day d can be picked up until day d + 3. Moving to a later day sends back every parcel past its last day, in the order they were dropped, and frees their compartments.
R6	Status shows free and total compartments for each size.
NF1	Codes must be random and hard to guess in production, but predictable in tests.
NF2	The rule for choosing a compartment may change (for example, keep LARGE ones for LARGE parcels).

Out of scope: several locker stations (Part C asks), returns (Part C asks), payment, the door hardware.

Commands and exact output

Command	Output
`COMPARTMENT id size`	`Compartment S1 (SMALL) added`
`DROP parcel customer size`	`P1 for Asha in S1 (SMALL), code 100001, pick up by day 4`
`PICKUP code`	`Asha picked up P1 from S1`
`DAY n`	`Day 5`, then one line per parcel sent back: `P3 returned to seller, M1 is free`
`STATUS`	`SMALL 1/2 free, MEDIUM 1/1 free, LARGE 0/1 free`
(errors)	`Compartment S1 already exists` · `Unknown size HUGE` · `Parcel P1 already exists` · `No compartment for P5 (LARGE)` · `Wrong code 999999` · `Too many wrong codes, try again tomorrow` · `Day cannot go back`

Part A — design

1	Classes and who owns what: locker, compartment, size, parcel, code.
2	R2: how do you find the right compartment? What makes "fits in" easy to write?
3	NF1: how do you make codes testable? You've seen the same idea with time.
4	Pickup gives you only a code. How do you find the parcel without scanning every compartment?

Part B — code

input.txt

```
COMPARTMENT S1 SMALL
COMPARTMENT S2 SMALL
COMPARTMENT M1 MEDIUM
COMPARTMENT L1 LARGE
COMPARTMENT S1 MEDIUM
COMPARTMENT X1 HUGE
DROP P1 Asha SMALL
DROP P2 Ravi SMALL
DROP P3 Meera SMALL
DROP P4 Kiran MEDIUM
DROP P5 Neha LARGE
DROP P1 Asha SMALL
STATUS
PICKUP 100002
PICKUP 100002
PICKUP 999999
PICKUP 123456
PICKUP 100001
DAY 2
PICKUP 100001
DROP P6 Neha SMALL
DAY 5
DROP P5 Neha LARGE
DAY 4
PICKUP 100003
STATUS
```

expected output

```
Compartment S1 (SMALL) added
Compartment S2 (SMALL) added
Compartment M1 (MEDIUM) added
Compartment L1 (LARGE) added
Compartment S1 already exists
Unknown size HUGE
P1 for Asha in S1 (SMALL), code 100001, pick up by day 4
P2 for Ravi in S2 (SMALL), code 100002, pick up by day 4
P3 for Meera in M1 (MEDIUM), code 100003, pick up by day 4
P4 for Kiran in L1 (LARGE), code 100004, pick up by day 4
No compartment for P5 (LARGE)
Parcel P1 already exists
SMALL 0/2 free, MEDIUM 0/1 free, LARGE 0/1 free
Ravi picked up P2 from S2
Wrong code 100002
Wrong code 999999
Wrong code 123456
Too many wrong codes, try again tomorrow
Day 2
Asha picked up P1 from S1
P6 for Neha in S1 (SMALL), code 100005, pick up by day 5
Day 5
P3 returned to seller, M1 is free
P4 returned to seller, L1 is free
P5 for Neha in L1 (LARGE), code 100006, pick up by day 8
Day cannot go back
Wrong code 100003
SMALL 1/2 free, MEDIUM 1/1 free, LARGE 0/1 free
```

Part C — pressure questions

1	**Change.** "Customers can return items through the locker: they drop the item, a courier collects it." What changes?
2	**Scale.** "500 lockers across Pune. The customer picks a locker at checkout, and a compartment must be waiting when the parcel arrives two days later."
3	**Security.** "Someone stands at the locker trying codes." What protects us?
4	**Failure.** "The code is right, but the door doesn't open."
5	**Defend it.** "Make SmallCompartment, MediumCompartment and LargeCompartment subclasses."

Problem 2 — Notification System

Core · 75 minutes (20 design + 55 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design a notification service. Other teams, like orders, payments and marketing, call it to notify users by email, SMS and push. Users choose what they get on which channel. Messages come from templates. Providers fail sometimes. Marketing messages mustn't wake people up at night. And if a caller retries the same request, the user must not get it twice."

After your questions, the requirements are

R1	A user has an email address, a phone number and a device, any of which may be missing.
R2	Templates have placeholders like `{order}`; `{name}` is filled in from the user's name. A missing value is refused.
R3	Categories: OTP always goes by SMS only, and users can't change that. ORDER goes on every channel by default; PROMO on none by default. A user can choose the channels for ORDER and PROMO. A message goes only on chosen channels the user has an address for.
R4	Each request has an ID from the caller. A request ID handled before is ignored, even if that first time had no channel or was held. Refused first, in this order: unknown user, unknown category, unknown template, missing value. A refused request doesn't use up its ID.
R5	Each channel is tried up to 3 times. A channel that fails 3 times is reported as failed; the other channels still go out.
R6	Quiet hours are 22:00 to 08:00. A PROMO message sent then is held, and goes out when the time is next set outside quiet hours. OTP and ORDER always go at once. The time starts at 09:00.
NF1	A new channel (WhatsApp) or a new provider must not change the sending code.
NF2	Many servers call at once, and callers retry: one request ID is sent once, however many times it arrives together (RUSH).
NF3	Sending must not slow callers down (Part C asks).

Out of scope: limits per user (Part C asks), different text per channel, delivery receipts, unsubscribe links.

Commands and exact output

Command	Output
`USER name email phone device`	`User Asha: EMAIL, SMS, PUSH` (`-` for a missing one)
`TEMPLATE id text…`	`Template shipped saved`
`PREF user category channels`	`Asha ORDER: EMAIL, PUSH` (channels separated by commas, or `NONE`)
`FAIL channel n`	`SMS will fail 2 times`: the next n sends on that channel fail
`TIME hh:mm`	`Time 08:00`, then any held messages that go out
`SEND request user category template key=value…`	one line per channel, in the order EMAIL, SMS, PUSH: `r1 EMAIL asha@mail.in: Hi Asha, order 5521 has shipped`, with `(attempt 3)` added after retries, or `r5 PUSH failed after 3 attempts`. Or one of: `r1 is a duplicate, ignored` · `r3 not sent: no channel for OTP` · `r8 held until 08:00 (quiet hours)`
`RUSH request user category template n key=value…`	the same request sent n times at the same moment: `Rush: 1 sent, 49 duplicates`
`(errors)`	`User Asha already exists` · `Unknown user Neha` · `Unknown category ALERT` · `Unknown template login` · `Missing value for {order}` · `Bad value order` · `Unknown channel FAX` · `OTP always goes by SMS`

Part A — design

1	Classes and who owns what: user, the user's choices, template, channel, provider, category.
2	NF1: which interface makes a new channel just a new class? Who decides which implementation is used?
3	R4 and NF2: where exactly does the duplicate check go, before or after the other checks? What makes it safe with many threads?
4	R5: where does retrying live, and why mustn't one channel's failure stop the others?

Part B — code

input.txt

```
USER Asha asha@mail.in 9822000001 asha-pixel
USER Ravi ravi@mail.in - -
USER Asha a@b.in - -
TEMPLATE shipped Hi {name}, order {order} has shipped
TEMPLATE otp {code} is your login code
TEMPLATE sale {name}, Diwali sale: {off}% off today
SEND r1 Asha ORDER shipped order=5521
SEND r1 Asha ORDER shipped order=5521
SEND r2 Ravi ORDER shipped
SEND r2 Ravi ORDER shipped order=77
SEND r3 Ravi OTP otp code=4821
SEND r4 Neha OTP otp code=4821
SEND r4 Asha ALERT otp code=4821
SEND r4 Asha OTP login code=4821
FAIL SMS 2
SEND r4 Asha OTP otp code=4821
FAIL PUSH 3
SEND r5 Asha ORDER shipped order=5530
PREF Asha ORDER EMAIL,PUSH
PREF Asha PROMO PUSH
PREF Ravi PROMO FAX
PREF Asha OTP EMAIL
SEND r6 Asha ORDER shipped order=5540
SEND r7 Ravi PROMO sale off=40
TIME 22:30
SEND r8 Asha PROMO sale off=40
SEND r9 Asha ORDER shipped order=5551
TIME 07:00
TIME 08:00
RUSH r20 Asha ORDER shipped 50 order=6000
SEND r20 Asha ORDER shipped order=6000
```

expected output

```
User Asha: EMAIL, SMS, PUSH
User Ravi: EMAIL
User Asha already exists
Template shipped saved
Template otp saved
Template sale saved
r1 EMAIL asha@mail.in: Hi Asha, order 5521 has shipped
r1 SMS 9822000001: Hi Asha, order 5521 has shipped
r1 PUSH asha-pixel: Hi Asha, order 5521 has shipped
r1 is a duplicate, ignored
Missing value for {order}
r2 EMAIL ravi@mail.in: Hi Ravi, order 77 has shipped
r3 not sent: no channel for OTP
Unknown user Neha
Unknown category ALERT
Unknown template login
SMS will fail 2 times
r4 SMS 9822000001: 4821 is your login code (attempt 3)
PUSH will fail 3 times
r5 EMAIL asha@mail.in: Hi Asha, order 5530 has shipped
r5 SMS 9822000001: Hi Asha, order 5530 has shipped
r5 PUSH failed after 3 attempts
Asha ORDER: EMAIL, PUSH
Asha PROMO: PUSH
Unknown channel FAX
OTP always goes by SMS
r6 EMAIL asha@mail.in: Hi Asha, order 5540 has shipped
r6 PUSH asha-pixel: Hi Asha, order 5540 has shipped
r7 not sent: no channel for PROMO
Time 22:30
r8 held until 08:00 (quiet hours)
r9 EMAIL asha@mail.in: Hi Asha, order 5551 has shipped
r9 PUSH asha-pixel: Hi Asha, order 5551 has shipped
Time 07:00
Time 08:00
r8 PUSH asha-pixel: Asha, Diwali sale: 40% off today
Rush: 1 sent, 49 duplicates
r20 is a duplicate, ignored
```

Part C — pressure questions

1	**Performance.** "SEND waits for three providers, with retries. The orders team says checkout got slow."
2	**Change.** "At most 2 promotions per user per day." You've built something like this before.
3	**Change.** "Add WhatsApp. Also, SMS text must fit in 160 characters, while email can be long."
4	**Failure.** "The SMS provider is down for an hour." Retrying 3 times doesn't help. What does?
5	**Defend it.** "Just make `send()` synchronized and use a plain HashSet. Simpler."

Problem 3 — Inventory Management

Core · 75 minutes (20 design + 55 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design the inventory system for an online store like Flipkart, with several warehouses. At checkout we reserve the stock. The reservation is confirmed when payment succeeds, or released if the customer cancels or doesn't pay in time. We choose the warehouse that ships the order, and alert purchasing when stock runs low. During a sale, we must never sell more than we have."

After your questions, the requirements are

R1	Warehouses have a name and a location on a grid; distance = |x1 − x2| + |y1 − y2| km. Products have a SKU and an alert level.
R2	For each warehouse and product: **on hand** and **reserved**; available = on hand − reserved. Restocking adds to on hand (at least 1).
R3	A reservation has a customer location and items (`SKU:qty`). All items come from one warehouse: the nearest one that has every item available (a tie goes to the name in alphabetical order). It holds the stock for 15 minutes. IDs are R1, R2, ... (refused attempts don't take a number). Refused, in this order, if: an item is badly written; item by item, the product is unknown or the quantity is below 1; no single warehouse has all the items.
R4	Confirming (payment done) takes the items out of on hand and reserved. Cancelling releases them. Only a RESERVED reservation can be confirmed or cancelled.
R5	Moving the clock expires every reservation whose hold has ended (at or before the new time), soonest first, then by number, and releases its stock. The time starts at 09:00.
R6	When a reservation makes a product's available stock in a warehouse drop from at or above its alert level to below it, purchasing is alerted: once per drop, printed after the result.
NF1	The warehouse choice will change (cheapest shipping, splitting an order).
NF2	Alerts may also go to email or a dashboard.
NF3	In a flash sale, many customers reserve the last units at the same moment: never sell more than there is (RUSH). Orders at different warehouses must not wait for each other.

Out of scope: splitting an order across warehouses (Part C asks), transfers between warehouses, returns, batches and expiry dates.

Commands and exact output

Command	Output
`WAREHOUSE name x y`	`Warehouse Pune at (0,0)`
`PRODUCT sku alert-level`	`Product KURTA, alert below 3`
`RESTOCK warehouse sku qty`	`Pune KURTA: 5 on hand, 5 available`
`TIME hh:mm`	`Time 10:15` , then one line per expired hold: `R2 expired, stock back at Mumbai`
`RESERVE x y sku:qty…`	`R2 at Mumbai (13 km): KURTA x1, MUG x2, hold until 10:15`
`CONFIRM / CANCEL id`	`R1 confirmed, ship from Pune` · `R3 cancelled, stock back at Pune`
`STOCK sku`	one line per warehouse that has stocked it, by name: `KURTA at Pune: 3 on hand, 0 reserved, 3 available`
`RUSH sku n x y`	n customers at (x,y) each reserve one unit at the same moment: `Rush: 13 reserved, 37 refused`
`(alert)`	`-> purchasing: KURTA at Pune is down to 2`
`(errors)`	`Unknown warehouse Delhi` · `Unknown product SOCKS` · `Quantity must be at least 1` · `Bad item KURTA` · `No warehouse has all the items` · `Unknown reservation R9` · `Cannot confirm R2: it is EXPIRED` (also `cancel`)

Part A — design

1	Entities and who owns what: product, warehouse, stock, reservation, the warehouse choice, the alert.
2	Why keep "reserved" separately, instead of taking the items off "on hand" at checkout?
3	NF3: what is the one indivisible step in a reservation, and what do you lock? An order can have several items.
4	R6: how do you alert once per drop, and not on every sale below the level?

Part B — code

input.txt

```
WAREHOUSE Pune 0 0
WAREHOUSE Mumbai 15 0
WAREHOUSE Nagpur 40 10
PRODUCT KURTA 3
PRODUCT MUG 2
RESTOCK Pune KURTA 5
RESTOCK Pune MUG 1
RESTOCK Mumbai KURTA 10
RESTOCK Mumbai MUG 6
RESTOCK Delhi KURTA 1
RESTOCK Pune SOCKS 1
TIME 10:00
RESERVE 2 0 KURTA:2
RESERVE 3 1 KURTA:1 MUG:2
RESERVE 1 1 KURTA:1
RESERVE 0 0 KURTA:0
RESERVE 0 0 KURTA
RESERVE 0 0 SOCKS:1
RESERVE 0 0 KURTA:20
TIME 10:10
RESERVE 1 0 MUG:1
CONFIRM R1
CONFIRM R1
CANCEL R3
STOCK KURTA
TIME 10:15
CONFIRM R2
CONFIRM R9
RESERVE 0 0 KURTA:1
TIME 10:30
RUSH KURTA 50 20 0
STOCK KURTA
```

expected output

```
Warehouse Pune at (0,0)
Warehouse Mumbai at (15,0)
Warehouse Nagpur at (40,10)
Product KURTA, alert below 3
Product MUG, alert below 2
Pune KURTA: 5 on hand, 5 available
Pune MUG: 1 on hand, 1 available
Mumbai KURTA: 10 on hand, 10 available
Mumbai MUG: 6 on hand, 6 available
Unknown warehouse Delhi
Unknown product SOCKS
Time 10:00
R1 at Pune (2 km): KURTA x2, hold until 10:15
R2 at Mumbai (13 km): KURTA x1, MUG x2, hold until 10:15
R3 at Pune (2 km): KURTA x1, hold until 10:15
-> purchasing: KURTA at Pune is down to 2
Quantity must be at least 1
Bad item KURTA
Unknown product SOCKS
No warehouse has all the items
Time 10:10
R4 at Pune (1 km): MUG x1, hold until 10:25
R1 confirmed, ship from Pune
Cannot confirm R1: it is CONFIRMED
R3 cancelled, stock back at Pune
KURTA at Mumbai: 10 on hand, 1 reserved, 9 available
KURTA at Pune: 3 on hand, 0 reserved, 3 available
Time 10:15
R2 expired, stock back at Mumbai
Cannot confirm R2: it is EXPIRED
Unknown reservation R9
R5 at Pune (0 km): KURTA x1, hold until 10:30
-> purchasing: KURTA at Pune is down to 2
Time 10:30
R4 expired, stock back at Pune
R5 expired, stock back at Pune
Rush: 13 reserved, 37 refused
-> purchasing: KURTA at Mumbai is down to 2
-> purchasing: KURTA at Pune is down to 2
KURTA at Mumbai: 10 on hand, 10 reserved, 0 available
KURTA at Pune: 3 on hand, 3 reserved, 0 available
```

Follow the numbers. Pune has 5 KURTA. R1 takes 2, leaving 3: not below 3, so no alert. R3 takes 1, leaving 2: it crosses the line, so one alert. After R3 is cancelled it's back at 3, so R5 crosses again and alerts again. Pune's MUG starts at 1, already below 2, so R4 never alerts.

Part C — pressure questions

1	**Change.** "If no warehouse has everything, split the order across warehouses."
2	**Scale.** "Stock lives in a database, and 20 app servers take orders."
3	**Failure.** "Payment succeeds, but the confirmation arrives after the hold expired."
4	**Performance.** "A new phone launch: 50,000 reservations a minute for one product at one warehouse."
5	**Defend it.** "Why expire holds at all? Keep them until the customer pays."

Problem 4 — Ride-sharing

Heavy · 100 minutes (25 design + 75 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design Uber or Ola. A rider asks for a ride in a vehicle type, from a pickup point to a drop point. We offer the ride to the nearest available driver of that type; the driver accepts or declines, and if they decline we try the next one. Show the fare up front, with surge pricing when demand is high. Riders can cancel, and rate the driver after the trip."

After your questions, the requirements are

R1	Drivers have a unique name, a vehicle type (AUTO, MINI, SEDAN) and a location on the grid (distance as in Problem 3). They start AVAILABLE. A driver can go offline only when AVAILABLE, and online only when OFFLINE.
R2	Fare = base + per km × distance from pickup to drop: AUTO Rs 25 + 10/km, MINI Rs 40 + 14/km, SEDAN Rs 60 + 18/km. **Surge:** count the rides of that type waiting for a driver's answer, plus this one, and the AVAILABLE drivers of that type within 5 km of the pickup. If the rides outnumber the drivers, the fare is ×1.5 (`fare * 150 / 100`). The fare is fixed at request time.
R3	Ride IDs are R1, R2, ... The ride is offered to the nearest AVAILABLE driver of that type within 5 km of the pickup (a tie goes to the name in alphabetical order), who is held for it. If there's none, the ride ends as NO_DRIVER. A rider can have only one ride at a time that hasn't ended.
R4	The offered driver accepts or declines. On a decline, that driver is free again, and the ride is offered to the next nearest driver who hasn't declined it; if there's none, NO_DRIVER.
R5	Life cycle: OFFERED → ACCEPTED → STARTED → COMPLETED. The rider can cancel while OFFERED (free) or ACCEPTED (Rs 50 fee). On completion the rider pays the fare, the driver gets 80% (rounded down), and the driver is now at the drop point and AVAILABLE.
R6	A completed ride can be rated once, 1 to 5 stars; the driver's average is shown.
NF1	Fare rules, surge and matching (highest rated, shortest arrival time) will change; none of them should touch the ride's life cycle.
NF2	Many riders request at the same moment: a driver is never offered two rides (RUSH), and requests near different drivers must not wait for each other.

Out of scope: payment, live GPS tracking, shared rides (Part C asks), offer timeouts (Part C asks), rating riders.

Commands and exact output

Command	Output
`DRIVER name type x y`	`Driver Kiran (MINI) at (1,1)`
`ONLINE / OFFLINE driver`	`Kiran is online` · `Kiran is offline`
`REQUEST rider type px py dx dy`	`R3: Neha, MINI, 4 km, Rs 144 (surge 1.5x), offered to Pooja (4 km away)` or `R5: no AUTO near (5,5)`
`ACCEPT / DECLINE ride`	`Meera accepted R1` · `Kiran declined R1, offered to Meera (3 km away)` or `Kiran declined R2, no other driver`
`START / COMPLETE ride`	`R1 started` · `R1 completed: Asha pays Rs 124, Meera gets Rs 99`
`CANCEL ride`	`R3 cancelled, fee Rs 50` or `R4 cancelled, no fee`
`RATE ride stars`	`Meera rated 5, now 4.5 from 2 ratings` (`1 rating` for one)
`RUSH type px py dx dy n`	n riders (Rider0, Rider1, ...) request at the same moment: `Rush: 3 got a driver, 47 found none`

Error	Output
`duplicates and unknowns`	`Driver Kiran already exists` · `Unknown driver Raj` · `Unknown vehicle type TAXI` · `Unknown ride R9`
`a second ride / a move the life cycle doesn't allow`	`Asha already has a ride (R1)` · `Cannot start R3: it is OFFERED` (also `accept` , `decline` , `complete` , `cancel` , `rate`)
`online / offline`	`Kiran is already online` · `Meera is ON_TRIP, cannot go offline`
`rating`	`R1 is already rated` · `Stars must be 1 to 5`

Part A — design

1	Actors, entities and who owns what. Draw the ride's life cycle and the driver's statuses.
2	NF1: which interfaces, and what does each one decide?
3	NF2: Food Delivery used one lock for all dispatch. What is the resource riders fight over here, and how can you claim it without one big lock?
4	R4: what must the ride remember so it isn't offered to the same driver again?

Part B — code

input.txt

```
DRIVER Kiran MINI 1 1
DRIVER Meera MINI 3 0
DRIVER Pooja MINI 4 4
DRIVER Imran MINI 9 9
DRIVER Sunil AUTO 0 1
DRIVER Joseph SEDAN 2 2
DRIVER Kiran AUTO 0 0
REQUEST Asha MINI 0 0 4 2
DECLINE R1
ACCEPT R1
REQUEST Ravi MINI 1 0 5 0
REQUEST Neha MINI 2 2 2 6
REQUEST Asha AUTO 0 0 1 1
REQUEST Dev TAXI 0 0 1 1
DECLINE R2
REQUEST Arjun SEDAN 0 0 10 0
REQUEST Kavya AUTO 5 5 6 6
START R1
START R3
ACCEPT R3
CANCEL R3
CANCEL R4
OFFLINE Meera
COMPLETE R1
RATE R1 4
RATE R1 5
RATE R3 5
RATE R9 5
OFFLINE Kiran
ONLINE Kiran
ONLINE Kiran
REQUEST Asha MINI 4 2 0 0
ACCEPT R6
START R6
COMPLETE R6
RATE R6 5
RUSH MINI 2 2 3 3 50
```

expected output

```
Driver Kiran (MINI) at (1,1)
Driver Meera (MINI) at (3,0)
Driver Pooja (MINI) at (4,4)
Driver Imran (MINI) at (9,9)
Driver Sunil (AUTO) at (0,1)
Driver Joseph (SEDAN) at (2,2)
Driver Kiran already exists
R1: Asha, MINI, 6 km, Rs 124, offered to Kiran (2 km away)
Kiran declined R1, offered to Meera (3 km away)
Meera accepted R1
R2: Ravi, MINI, 4 km, Rs 96, offered to Kiran (1 km away)
R3: Neha, MINI, 4 km, Rs 144 (surge 1.5x), offered to Pooja (4 km away)
Asha already has a ride (R1)
Unknown vehicle type TAXI
Kiran declined R2, no other driver
R4: Arjun, SEDAN, 10 km, Rs 240, offered to Joseph (4 km away)
R5: no AUTO near (5,5)
R1 started
Cannot start R3: it is OFFERED
Pooja accepted R3
R3 cancelled, fee Rs 50
R4 cancelled, no fee
Meera is ON_TRIP, cannot go offline
R1 completed: Asha pays Rs 124, Meera gets Rs 99
Meera rated 4, now 4.0 from 1 rating
R1 is already rated
Cannot rate R3: it is CANCELLED
Unknown ride R9
Kiran is offline
Kiran is online
Kiran is already online
R6: Asha, MINI, 6 km, Rs 124, offered to Meera (0 km away)
Meera accepted R6
R6 started
R6 completed: Asha pays Rs 124, Meera gets Rs 99
Meera rated 5, now 4.5 from 2 ratings
Rush: 3 got a driver, 47 found none
```

Check the surge by hand. When Neha asks for a MINI at (2,2), R2 is still waiting for Kiran's answer, so 2 rides are searching. Free MINI drivers within 5 km: only Pooja (Kiran is held for R2, Meera is on R1, Imran is 14 km away). 2 > 1, so the 4 km fare of 40 + 56 = 96 becomes 144.

Part C — pressure questions

1	**Change.** "If the driver doesn't answer in 15 seconds, treat it as a decline."
2	**Scale.** "A million drivers send their location every 4 seconds. Finding the nearest must stay fast."
3	**Change.** "Shared rides: two riders going the same way share one car."
4	**Failure.** "The driver's app crashes in the middle of a trip."
5	**Defend it.** "Surge is just a multiplier. Put an `if` in the fare code instead of an interface."

Hints

Read one level at a time, and only after 15 minutes of being stuck. Each level gives away more than the last.

Problem 1 — Amazon Locker

Hint 1 (a nudge). It's a small parking lot: spots of different sizes, a "smallest that fits" rule, and a ticket (here, a code). The code is the only thing the customer brings.

Hint 2 (the shape). `Size` enum, declared smallest first · `Compartment` (id, size, its parcel or null) · `Parcel` record (id, customer, code, compartment, last day) · `CodeGenerator` interface · `Locker` with compartments in the order added and a map from code to parcel.

Hint 3 (the key decision). Fits = `compartment.size.compareTo(parcel size) >= 0`; take the minimum by size among free ones that fit. Pickup = remove the code from the map; if nothing was there, count a wrong try. A new day resets the count and sends back expired parcels.

Problem 2 — Notification System

Hint 1. Separate *which channels* (category defaults plus user choices plus addresses), *how to send* on one channel (an interface per provider), and the service's own rules (duplicates, retries, quiet hours).

Hint 2. `Channel` and `Category` enums (the category knows its defaults) · `User` (addresses, choices) · `Template` with `render` · `ChannelSender` interface with a fake provider that can be told to fail · `NotificationService` holding a `Map<Channel, ChannelSender>`, a concurrent set of request IDs and a held list.

Hint 3. Validate and render first, so a bad request doesn't use up its ID. Then `if (!seen.add(id))`: a duplicate. `add` on a concurrent set is one atomic step; `contains` followed by `add` is two. Retry inside a loop per channel, catching that channel's failure only.

Problem 3 — Inventory Management

Hint 1. Three numbers, not one: on hand, reserved, available. Every operation moves stock between them, and each operation touches one warehouse.

Hint 2. `Stock` (on hand, reserved) · `Warehouse` (location, stock by SKU) · `Reservation` (warehouse, items, expiry, status) · `FulfilmentStrategy` that ranks warehouses · `StockListener` for alerts · `Inventory` as the front door, with an injected time.

Hint 3. For each warehouse, best first: `synchronized (warehouse)` { if every item is available, reserve every item and create the reservation }; otherwise move on. Confirm, cancel and expiry lock the reservation's warehouse too. Alert when `before >= level && after < level`.

Problem 4 — Ride-sharing

Hint 1. The ride and the driver each have their own statuses, and they change together. Riders compete for drivers, so a driver must be "claimed" in one atomic step.

Hint 2. `VehicleType` enum with base and per-km prices · `Driver` with an `AtomicReference<DriverStatus>` · `Ride` with `RideStatus.canMoveTo` and a set of drivers who declined · `SurgePolicy` and `MatchingStrategy` interfaces · `RideService`.

Hint 3. Matching returns candidates, nearest first; for each, `status.compareAndSet(AVAILABLE, OFFERED)`. The first success is your driver; a failure means someone faster got them, so try the next. Every ride change happens inside `synchronized (ride)`; claiming a driver isn't a lock, so locks never nest.

Solution 1 — Amazon Locker

Questions a strong candidate asks first

Question	Answer assumed here
Which sizes? Can a small parcel use a bigger compartment?	Three sizes; yes, the smallest free one that fits
How is the code made and sent?	6 digits, from a generator we're given; SMS is someone else's job
How long do we keep a parcel? Then what?	3 days after the drop day; then back to the seller
Any protection against guessing codes?	3 wrong codes in a row lock the screen until tomorrow
One locker or many?	One station here (Part C 2 asks about many)

Part A — model answer

```
Locker 1 ◆———— * Compartment (id, Size, parcel or null)          Size: enum SMALL < MEDIUM < LARGE
Locker ————> Map<String, Parcel> byCode (in drop order)          Parcel: record (id, customer, code,
Locker ————> «interface» CodeGenerator  ◁- - -  SequenceCodes                  compartment, lastDay)
```

1. Who owns what. The **locker** owns its compartments, the code index, today's date and the wrong-code count. A **compartment** knows which parcel is in it. A **parcel** is a record: its code and last day never change. The **code generator** owns how codes are made.

2. "Fits in" is a comparison. Declare the enum smallest first, and "fits" becomes `c.size.compareTo(size) >= 0`. Among free compartments that fit, take the minimum by size. Java's `Stream.min` keeps the first of equal elements, so within a size you get the one added first. A different rule (NF2) only changes `findFree`; with more rules, it becomes a strategy.

3. Codes are passed in, like the clock. In production, `SecureRandom` makes 6-digit codes; in the test, `SequenceCodes` gives 100001, 100002, ... The locker never knows which. The loop `do { code = codes.next(); } while (taken)` keeps codes unique even with random ones.

4. Look up by code. A `Map<code, Parcel>` makes pickup one lookup. Removing the entry is how a code stops working, so a used or expired code can never open anything. A `LinkedHashMap` also gives the drop order for sending parcels back.

In the interview: write this first. (1) `Size`, `Compartment`, `Parcel`, `CodeGenerator`: 10 minutes. (2) `drop` with `findFree`. (3) `pickUp` with the wrong-code count. (4) `advanceTo`. (5) Status. The domain code here is about 90 lines.

Part B — the code

```
import java.util.*;

// Declared smallest first, so compareTo() means "fits in"
enum Size { SMALL, MEDIUM, LARGE }

class Compartment {
    final String id;
    final Size size;
    Parcel parcel;                                          // null when free

    Compartment(String id, Size size) { this.id = id; this.size = size; }
}

record Parcel(String id, String customer, String code, Compartment compartment, int lastDay) {}

interface CodeGenerator {
    String next();
}

// For the test: predictable codes. Production would use SecureRandom.
class SequenceCodes implements CodeGenerator {
    private int last = 100000;
    public String next() { return String.valueOf(++last); }
}

class Locker {
    static final int KEEP_DAYS = 3, MAX_WRONG = 3;
    private final Map<String, Compartment> compartments = new LinkedHashMap<>();
    private final Map<String, Parcel> byCode = new LinkedHashMap<>();    // drop order
    private final Set<String> parcelIds = new HashSet<>();
    private final CodeGenerator codes;
    private int today = 1, wrongInARow;

    Locker(CodeGenerator codes) { this.codes = codes; }

    void addCompartment(String id, Size size) {
        if (compartments.containsKey(id))
            throw new IllegalArgumentException("Compartment " + id + " already exists");
        compartments.put(id, new Compartment(id, size));
    }

    Parcel drop(String id, String customer, Size size) {
        if (parcelIds.contains(id))
            throw new IllegalArgumentException("Parcel " + id + " already exists");
        Compartment c = findFree(size).orElseThrow(
                () -> new IllegalStateException("No compartment for " + id + " (" + size + ")"));
        String code;
        do { code = codes.next(); } while (byCode.containsKey(code));
        Parcel p = new Parcel(id, customer, code, c, today + KEEP_DAYS);
```

```
        c.parcel = p;
        byCode.put(code, p);
        parcelIds.add(id);
        return p;
    }

    // The smallest size that fits; within a size, the first added
    private Optional<Compartment> findFree(Size size) {
        return compartments.values().stream()
                .filter(c -> c.parcel == null && c.size.compareTo(size) >= 0)
                .min(Comparator.comparing(c -> c.size));
    }

    Parcel pickUp(String code) {
        if (wrongInARow >= MAX_WRONG)
            throw new IllegalStateException("Too many wrong codes, try again tomorrow");
        Parcel p = byCode.remove(code);
        if (p == null) {
            wrongInARow++;
            throw new IllegalArgumentException("Wrong code " + code);
        }
        wrongInARow = 0;
        p.compartment().parcel = null;
        return p;
    }

    // Moves to a new day; returns the parcels sent back to the seller.
    List<Parcel> advanceTo(int day) {
        if (day < today) throw new IllegalArgumentException("Day cannot go back");
        today = day;
        wrongInARow = 0;
        List<Parcel> expired = new ArrayList<>();
        Iterator<Parcel> it = byCode.values().iterator();
        while (it.hasNext()) {
            Parcel p = it.next();
            if (p.lastDay() < day) {
                it.remove();
                p.compartment().parcel = null;
                expired.add(p);
            }
        }
        return expired;
    }

    String status() {
        List<String> parts = new ArrayList<>();
        for (Size s : Size.values()) {
            List<Compartment> ofSize = compartments.values().stream().filter(c -> c.size == s).toList();
            long free = ofSize.stream().filter(c -> c.parcel == null).count();
            parts.add(s + " " + free + "/" + ofSize.size() + " free");
        }
        return String.join(", ", parts);
    }
}

public class Main {
    public static void main(String[] args) {
        Locker locker = new Locker(new SequenceCodes());
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "COMPARTMENT" -> {
                        locker.addCompartment(p[1], size(p[2]));
                        System.out.println("Compartment " + p[1] + " (" + p[2] + ") added");
                    }
                    case "DROP" -> {
                        Parcel x = locker.drop(p[1], p[2], size(p[3]));
                        Compartment c = x.compartment();
                        System.out.println(x.id() + " for " + x.customer() + " in " + c.id + " ("
                                + c.size + "), code " + x.code() + ", pick up by day " + x.lastDay());
                    }
                    case "PICKUP" -> {
                        Parcel x = locker.pickUp(p[1]);
                        System.out.println(x.customer() + " picked up " + x.id() + " from "
                                + x.compartment().id);
                    }
                    case "DAY" -> {
                        List<Parcel> back = locker.advanceTo(Integer.parseInt(p[1]));
                        System.out.println("Day " + p[1]);
                        for (Parcel x : back)
                            System.out.println(x.id() + " returned to seller, " + x.compartment().id
                                    + " is free");
                    }
                    case "STATUS" -> System.out.println(locker.status());
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    static Size size(String s) {
        try {
            return Size.valueOf(s);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Unknown size " + s);
```

```
        }
    }
}
```

How to read it: `pickUp` checks the lock before looking at the code, so a correct code is refused too once locked; that's the point. `advanceTo` removes with the iterator, the safe way to delete while looping over a map.

Part C — model answers

1. Returns. The same compartments, used in the other direction. The parcel gets a direction, DELIVERY or RETURN. For a return, the customer gets a drop code and the courier a pickup code. `findFree`, the code map and expiry stay the same.

2. Many lockers, reserved at checkout. A compartment gets a third status: FREE → RESERVED (for an order, with an end time) → OCCUPIED. At checkout, the order reserves a compartment of the right size; on arrival, the agent drops into that one. Unused reservations are released by a scheduled job, like BookMyShow's seat holds. Each locker's state lives in a database, and a reservation is a conditional update (`WHERE status = 'FREE'`).

3. Guessing codes. Random 6-digit codes from `SecureRandom` (one in a million per try), R4's lock after 3 wrong tries, codes that only work at one locker and die with the parcel, and an alert to operations after repeated lockouts.

4. The door doesn't open. Change state only after the hardware confirms. If the door fails, keep the parcel and its code as they are, mark the compartment OUT_OF_SERVICE so `findFree` skips it, and alert maintenance. If you freed the compartment first, the customer would lose a parcel that's still inside.

5. "Make subclasses per size." "You'd be right if the sizes behaved differently. They don't: each one opens, holds a parcel and frees the same way, and only the size differs. That's data, so an enum, and `compareTo` gives 'fits in' for free. With three subclasses I'd need `instanceof` to compare them. If a refrigerated compartment with its own rules comes along, that one gets a class."

Common mistakes
- Exact size only: refusing a SMALL parcel while a MEDIUM compartment is free.
- Finding a parcel by scanning all compartments for its code.
- Calling `new Random()` inside the locker, so the test can't know the codes.
- Freeing the compartment but leaving the code in the map.
- Letting a correct code through after 3 wrong ones.

Score yourself (1 point each)

☐	Size is an enum, declared smallest first	☐	The smallest free compartment that fits
☐	The code generator is passed in	☐	Pickup is one lookup by code
☐	Codes are unique among parcels inside	☐	Three wrong codes lock until tomorrow
☐	Expired parcels go back in drop order	☐	Your output matches exactly
☐	Part C 2: a RESERVED status with a hold	☐	Part C 4: change state after the door opens

Solution 2 — Notification System

Questions a strong candidate asks first

Question	Answer assumed here
Which channels? Who chooses them?	Email, SMS, push; category defaults plus the user's choices
Are some messages special?	OTP: SMS only, always at once. PROMO: opt-in, quiet hours
Can the same request arrive twice?	Yes: callers retry, so each request carries an ID
What if a provider fails?	Up to 3 tries per channel; the other channels still go
Must the caller wait for delivery?	Here yes, to keep the test simple; Part C 1 makes it async

Part A — model answer

```
NotificationService ———> * User (addresses by channel, choices by category)
NotificationService ———> * Template (render)
NotificationService ———> Map<Channel, «interface» ChannelSender>  ◁- - -  FakeProvider (real: SES, Twilio, FCM)
NotificationService ———> Set<String> seenRequests (concurrent),  List<Held> held
Channel: enum   Category: enum with defaults()   Delivery, SendResult: records
```

1. Who owns what. A **user** owns their addresses and choices. A **category** owns its defaults. A **template** owns rendering. A **sender** owns talking to one provider. The **service** owns the rules: which channels, duplicates, retries and quiet hours.

2. One interface per channel (NF1). `ChannelSender` has one method, `send(address, text)`. The service is given a map from channel to sender, so it never knows about SES or Twilio. WhatsApp = a new enum value and a new sender class; the delivery loop doesn't change. Switching SMS providers = a different object in the map.

3. Check everything, then the duplicate check, in one atomic step. Validation and rendering come first, so a typo doesn't use up the ID and the caller can fix it and retry. Then `seenRequests.add(id)` on a concurrent set: of 50 threads with the same ID, exactly one gets `true`. Tested: with `contains` then `add` instead, RUSH sent the same request twice in **2 runs out of 260**. Rare is the danger: tests pass, and a user gets two OTPs in production.

4. Retry per channel, inside the service. Each channel gets its own loop and its own count. A failure is caught for that channel only, so a dead push service never blocks email. Quiet hours: the check and the add to `held` are inside the same lock as `setTime`, so a message can't slip in just after the held list was released.

In the interview: write this first. (1) The enums, `User`, `Template.render`: 15 minutes. (2) `ChannelSender` and the fake provider. (3) `send`: channels and the duplicate check. (4) `deliver` with retries. (5) Quiet hours, then RUSH. The domain code here is about 135 lines.

Part B — the code

```
import java.time.LocalTime;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.IntPredicate;
import java.util.regex.*;

enum Channel { EMAIL, SMS, PUSH }

enum Category {
    OTP, ORDER, PROMO;

    // Used when the user hasn't chosen
    Set<Channel> defaults() {
        return switch (this) {
            case OTP -> EnumSet.of(Channel.SMS);
            case ORDER -> EnumSet.allOf(Channel.class);
            case PROMO -> EnumSet.noneOf(Channel.class);
        };
    }
}

class User {
    final String name;
    final Map<Channel, String> addresses;                       // only the ones the user has
    final Map<Category, Set<Channel>> choices = new ConcurrentHashMap<>();

    User(String name, Map<Channel, String> addresses) { this.name = name; this.addresses = addresses; }
}

class Template {
    private static final Pattern PLACEHOLDER = Pattern.compile("\\{(\\w+)\\}");
    final String text;

    Template(String text) { this.text = text; }

    String render(Map<String, String> values) {
        return PLACEHOLDER.matcher(text).replaceAll(m -> {
            String v = values.get(m.group(1));
            if (v == null) throw new IllegalArgumentException("Missing value for {" + m.group(1) + "}");
            return Matcher.quoteReplacement(v);
        });
    }
}
```

```
class DeliveryException extends RuntimeException {}

interface ChannelSender {
    void send(String address, String text);                    // throws DeliveryException
}

// Stands in for an email, SMS or push provider; can be told to fail
class FakeProvider implements ChannelSender {
    private final AtomicInteger failuresLeft = new AtomicInteger();

    void failNext(int n) { failuresLeft.set(n); }

    public void send(String address, String text) {
        if (failuresLeft.getAndUpdate(n -> Math.max(0, n - 1)) > 0) throw new DeliveryException();
    }
}

record Delivery(String request, Channel channel, String address, String text, int attempts) {
    boolean sent() { return attempts > 0; }                    // 0 = every attempt failed
}

enum Outcome { SENT, DUPLICATE, NO_CHANNEL, HELD }

record SendResult(Outcome outcome, List<Delivery> deliveries) {}

class NotificationService {
    static final int MAX_ATTEMPTS = 3;
    static final LocalTime QUIET_FROM = LocalTime.of(22, 0), QUIET_TO = LocalTime.of(8, 0);
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Template> templates = new ConcurrentHashMap<>();
    private final Map<Channel, ChannelSender> senders;
    private final Set<String> seenRequests = ConcurrentHashMap.newKeySet();
    private final List<Held> held = new ArrayList<>();         // guarded by itself
    private LocalTime now = LocalTime.of(9, 0);

    private record Held(String request, User user, Set<Channel> channels, String text) {}

    NotificationService(Map<Channel, ChannelSender> senders) { this.senders = senders; }

    void addUser(String name, Map<Channel, String> addresses) {
        if (users.putIfAbsent(name, new User(name, addresses)) != null)
            throw new IllegalArgumentException("User " + name + " already exists");
    }

    void addTemplate(String id, String text) { templates.put(id, new Template(text)); }

    void choose(String userName, Category category, Set<Channel> channels) {
        if (category == Category.OTP) throw new IllegalArgumentException("OTP always goes by SMS");
        user(userName).choices.put(category, channels);
    }

    SendResult send(String request, String userName, Category category, String templateId,
                    Map<String, String> values) {
        User u = user(userName);
        Template t = templates.get(templateId);
        if (t == null) throw new IllegalArgumentException("Unknown template " + templateId);
        Map<String, String> all = new HashMap<>(values);
        all.putIfAbsent("name", u.name);
        String text = t.render(all);

        // Atomic: of many calls with the same ID, exactly one gets past this line
        if (!seenRequests.add(request)) return new SendResult(Outcome.DUPLICATE, List.of());
        Set<Channel> channels = EnumSet.noneOf(Channel.class);
        channels.addAll(u.choices.getOrDefault(category, category.defaults()));
        channels.retainAll(u.addresses.keySet());
        if (channels.isEmpty()) return new SendResult(Outcome.NO_CHANNEL, List.of());
        if (category == Category.PROMO) {
            synchronized (held) {
                if (isQuiet(now)) {
                    held.add(new Held(request, u, channels, text));
                    return new SendResult(Outcome.HELD, List.of());
                }
            }
        }
        return new SendResult(Outcome.SENT, deliver(request, u, channels, text));
    }

    // Moves the clock; outside quiet hours, held messages go out. Returns their deliveries.
    List<Delivery> setTime(LocalTime t) {
        synchronized (held) {
            now = t;
            if (isQuiet(t)) return List.of();
            List<Delivery> out = new ArrayList<>();
            for (Held h : held) out.addAll(deliver(h.request(), h.user(), h.channels(), h.text()));
            held.clear();
            return out;
        }
    }

    private static boolean isQuiet(LocalTime t) {
        return !t.isBefore(QUIET_FROM) || t.isBefore(QUIET_TO);
    }

    // Each channel separately; one channel failing doesn't stop the others
    private List<Delivery> deliver(String request, User u, Set<Channel> channels, String text) {
        List<Delivery> out = new ArrayList<>();
        for (Channel ch : channels) {
            String to = u.addresses.get(ch);
            int attempts = 0;
            for (int i = 1; i <= MAX_ATTEMPTS && attempts == 0; i++) {
```

```
                try {
                    senders.get(ch).send(to, text);
                    attempts = i;
                } catch (DeliveryException e) {
                    // try again (a real system would wait 1 s, 2 s, 4 s ...)
                }
            }
            out.add(new Delivery(request, ch, to, text, attempts));
        }
        return out;
    }

    private User user(String name) {
        User u = users.get(name);
        if (u == null) throw new IllegalArgumentException("Unknown user " + name);
        return u;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Map<Channel, FakeProvider> providers = new EnumMap<>(Channel.class);
        for (Channel c : Channel.values()) providers.put(c, new FakeProvider());
        NotificationService service = new NotificationService(new EnumMap<>(providers));
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "USER" -> {
                        Map<Channel, String> addresses = new EnumMap<>(Channel.class);
                        for (Channel c : Channel.values())
                            if (!p[2 + c.ordinal()].equals("-")) addresses.put(c, p[2 + c.ordinal()]);
                        service.addUser(p[1], addresses);
                        System.out.println("User " + p[1] + ": " + names(addresses.keySet()));
                    }
                    case "TEMPLATE" -> {
                        service.addTemplate(p[1], line.split("\\s+", 3)[2]);
                        System.out.println("Template " + p[1] + " saved");
                    }
                    case "PREF" -> {
                        Set<Channel> chosen = EnumSet.noneOf(Channel.class);
                        if (!p[3].equals("NONE"))
                            for (String c : p[3].split(",")) chosen.add(channel(c));
                        service.choose(p[1], category(p[2]), chosen);
                        System.out.println(p[1] + " " + p[2] + ": " + names(chosen));
                    }
                    case "FAIL" -> {
                        providers.get(channel(p[1])).failNext(Integer.parseInt(p[2]));
                        System.out.println(p[1] + " will fail " + p[2] + " times");
                    }
                    case "TIME" -> {
                        List<Delivery> out = service.setTime(LocalTime.parse(p[1]));
                        System.out.println("Time " + p[1]);
                        out.forEach(Main::print);
                    }
                    case "SEND" -> {
                        SendResult r = service.send(p[1], p[2], category(p[3]), p[4], values(p, 5));
                        switch (r.outcome()) {
                            case SENT -> r.deliveries().forEach(Main::print);
                            case DUPLICATE -> System.out.println(p[1] + " is a duplicate, ignored");
                            case NO_CHANNEL ->
                                    System.out.println(p[1] + " not sent: no channel for " + p[3]);
                            case HELD -> System.out.println(p[1] + " held until 08:00 (quiet hours)");
                        }
                    }
                    // The same request ID sent n times at the same moment
                    case "RUSH" -> {
                        int n = Integer.parseInt(p[5]);
                        Map<String, String> v = values(p, 6);
                        int sent = rush(n, i -> service.send(p[1], p[2], category(p[3]), p[4], v)
                                .outcome() != Outcome.DUPLICATE);
                        System.out.println("Rush: " + sent + " sent, " + (n - sent) + " duplicates");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    static void print(Delivery d) {
        if (!d.sent())
            System.out.println(d.request() + " " + d.channel() + " failed after "
                    + NotificationService.MAX_ATTEMPTS + " attempts");
        else
            System.out.println(d.request() + " " + d.channel() + " " + d.address() + ": " + d.text()
                    + (d.attempts() > 1 ? " (attempt " + d.attempts() + ")" : ""));
    }

    static Map<String, String> values(String[] p, int from) {
        Map<String, String> v = new HashMap<>();
        for (int i = from; i < p.length; i++) {
            String[] kv = p[i].split("=", 2);
            if (kv.length != 2) throw new IllegalArgumentException("Bad value " + p[i]);
            v.put(kv[0], kv[1]);
        }
        return v;
```

```
    }

    static String names(Set<Channel> channels) {
        if (channels.isEmpty()) return "none";
        return String.join(", ", channels.stream().map(Enum::name).toList());
    }

    static Channel channel(String s) {
        try {
            return Channel.valueOf(s);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Unknown channel " + s);
        }
    }

    static Category category(String s) {
        try {
            return Category.valueOf(s);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Unknown category " + s);
        }
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

How to read it: `render` uses `replaceAll` with a function, so each `{key}` is looked up once and a value containing `$` or `{` stays as written. `EnumSet` keeps channels in declaration order, which fixes the output order. `Delivery.attempts` is 0 when every try failed.

Part C — model answers

1. Make it asynchronous. `send` validates, does the duplicate check, puts one job per channel on a queue (Kafka, SQS) and returns "accepted" at once. Workers per channel take jobs, send, and retry with growing waits (1 s, 2 s, 4 s). After the last try, the job goes to a dead-letter queue for a person to look at. Callers can check status by request ID.

2. Two promotions a day. A counter per user per date for PROMO, checked after the duplicate check so a retried request doesn't count twice. It's the fixed-window limiter from PDF 3, with a day as the window and the user as the key.

3. WhatsApp, and short SMS. WhatsApp is a new `Channel` value and a new sender. For length, keep templates per channel (`shipped.sms` , `shipped.email`) and fall back to the default one. Don't let a sender cut text: a cut OTP message is worse than none.

4. A provider is down for an hour. A **circuit breaker**: after, say, 10 failures in a row, stop calling that provider for a minute and fail fast, then try one call to see if it's back. Meanwhile, a `FailoverSender` (a decorator holding two SMS providers) switches to the second one. OTPs need that; promotions can just wait in the queue.

5. "synchronized send() and a HashSet." "You'd be right for one server at low volume. But a synchronized `send` holds the lock while it calls providers and retries, so every caller waits behind every other caller's SMS. Only the duplicate check needs to be atomic, and a concurrent set's `add` is atomic on its own. With several servers, the set moves to Redis (`SET key NX` with an expiry) or a unique key in the database."

Common mistakes

- Recording the ID before validating, so a request with a typo can never be sent.
- `if (!seen.contains(id)) { seen.add(id); … }` : two threads both pass the check.
- One channel's exception escaping the loop and stopping the other channels.
- Holding OTPs for quiet hours, or letting users switch OTPs off.
- `if (channel == EMAIL) … else if (channel == SMS) …` inside the service.

Score yourself (1 point each)

☐	One sender interface, a map by channel	☐	Category defaults plus user choices
☐	Only channels the user has an address for	☐	Validate before the duplicate check
☐	The duplicate check is one atomic add	☐	Retries per channel, failures isolated
☐	PROMO held in quiet hours, released later	☐	Your output matches exactly, RUSH included
☐	Part C 1: a queue and workers	☐	Part C 4: circuit breaker and failover

Solution 3 — Inventory Management

Questions a strong candidate asks first

Question	Answer assumed here
Can one order be shipped from several warehouses?	Not now: one warehouse per order (Part C 1)
How is the warehouse chosen?	The nearest one that has everything
How long is stock held at checkout?	15 minutes, then it's released
When exactly does purchasing get alerted?	When a warehouse's available stock drops below the level
Is stock counted per warehouse or in total?	Per warehouse; that's what can actually be shipped

Part A — model answer

```
Inventory ———> * Warehouse (location, Map<sku, Stock>)          Stock: onHand, reserved → available()
Inventory ———> * Reservation (warehouse, items, expiresAt, Status)  Product: record (sku, alertBelow)
Inventory ———> «interface» FulfilmentStrategy  ◁- - -  NearestWarehouse
Inventory ———> * «interface» StockListener     ◁- - -  PurchasingAlert

RESERVED ——confirm——> CONFIRMED      RESERVED ——cancel——> CANCELLED      RESERVED ——hold ends——> EXPIRED
```

1. Who owns what. A **warehouse** owns its stock and its lock. A **reservation** remembers its warehouse, so confirm and cancel know where to go. The **strategy** owns the order in which warehouses are tried; **listeners** own what happens on low stock. `Inventory` is the front door and owns the clock.

2. Three numbers. **On hand** is what's on the shelf; **reserved** is promised to checkouts that haven't paid; **available** is what can still be sold. If checkout took items off on hand, a cancelled or expired checkout would have to put them back, and the count would stop matching the shelf in between. With reserved kept apart, cancel and expiry are one subtraction, and confirm takes from both. Real systems call available "available to promise".

3. One warehouse, one lock (NF3). All items of an order come from one warehouse, so one lock covers them: `synchronized (warehouse) {` check every item; reserve every item `}`. That's all or nothing, with no lock ordering to worry about, and Mumbai never waits for Pune. Confirm, cancel and expiry take the same lock, so a confirm and an expiry can't both win. Tested: without the lock, RUSH reserved 14 or 15 units when only 13 existed, in **18 runs out of 120**.

4. Alert on the crossing. Compare before and after: `before >= level && after < level`. No flag to store or reset: once stock climbs back (a restock, a cancel), the next drop alerts again. Alerts are an Observer (NF2): purchasing today, a dashboard tomorrow.

In the interview: write this first. (1) `Stock`, `Warehouse`, `Reservation`: 15 minutes. (2) `reserve` with the warehouse lock. (3) Confirm and cancel. (4) Expiry in `setTime`. (5) Alerts, the report, then RUSH. The domain code here is about 190 lines; if time is short, make the strategy a private method and skip the report, and say where they'd go.

Part B — the code

```
import java.time.LocalTime;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.IntPredicate;

record Location(int x, int y) {
    int distanceTo(Location o) { return Math.abs(x - o.x) + Math.abs(y - o.y); }
    @Override public String toString() { return "(" + x + "," + y + ")"; }
}

record Product(String sku, int alertBelow) {}

class Stock {
    int onHand, reserved;
    int available() { return onHand - reserved; }
}

// Its stock is guarded by its own lock: synchronized (warehouse)
class Warehouse {
    final String name;
    final Location location;
    final Map<String, Stock> stock = new HashMap<>();           // sku -> stock

    Warehouse(String name, Location location) { this.name = name; this.location = location; }

    Stock of(String sku) { return stock.computeIfAbsent(sku, k -> new Stock()); }

    int available(String sku) {
        Stock s = stock.get(sku);
        return s == null ? 0 : s.available();
    }
}

enum Status { RESERVED, CONFIRMED, CANCELLED, EXPIRED }

// Its status only changes under its warehouse's lock
class Reservation {
    final int number;
    final String id;
    final Warehouse warehouse;
    final Map<String, Integer> items;
    final LocalTime expiresAt;
```

```
    Status status = Status.RESERVED;

    Reservation(int number, Warehouse w, Map<String, Integer> items, LocalTime expiresAt) {
        this.number = number; this.id = "R" + number;
        this.warehouse = w; this.items = items; this.expiresAt = expiresAt;
    }
}

interface FulfilmentStrategy {
    List<Warehouse> rank(Location customer, Collection<Warehouse> all);    // best first
}

class NearestWarehouse implements FulfilmentStrategy {
    public List<Warehouse> rank(Location customer, Collection<Warehouse> all) {
        return all.stream()
                .sorted(Comparator.comparingInt((Warehouse w) -> w.location.distanceTo(customer))
                        .thenComparing(w -> w.name))
                .toList();
    }
}

interface StockListener {
    void lowStock(Warehouse w, String sku, int available);
}

// Collects alerts; the driver prints them
class PurchasingAlert implements StockListener {
    private final List<String> outbox = Collections.synchronizedList(new ArrayList<>());

    public void lowStock(Warehouse w, String sku, int available) {
        outbox.add("-> purchasing: " + sku + " at " + w.name + " is down to " + available);
    }

    List<String> drain() {
        synchronized (outbox) {
            List<String> out = new ArrayList<>(outbox);
            outbox.clear();
            return out;
        }
    }
}

class Inventory {
    static final int HOLD_MINUTES = 15;
    private final Map<String, Warehouse> warehouses = new ConcurrentHashMap<>();
    private final Map<String, Product> products = new ConcurrentHashMap<>();
    private final Map<String, Reservation> reservations = new ConcurrentHashMap<>();
    private final AtomicInteger count = new AtomicInteger();
    private final FulfilmentStrategy strategy;
    private final List<StockListener> listeners;
    private volatile LocalTime now = LocalTime.of(9, 0);

    Inventory(FulfilmentStrategy strategy, List<StockListener> listeners) {
        this.strategy = strategy; this.listeners = listeners;
    }

    void addWarehouse(String name, Location l) { warehouses.put(name, new Warehouse(name, l)); }
    void addProduct(String sku, int alertBelow) { products.put(sku, new Product(sku, alertBelow)); }

    Stock restock(String warehouse, String sku, int qty) {
        Warehouse w = warehouse(warehouse);
        product(sku);
        if (qty < 1) throw new IllegalArgumentException("Quantity must be at least 1");
        synchronized (w) {
            w.of(sku).onHand += qty;
            return w.of(sku);
        }
    }

    // All items from one warehouse, or nothing
    Reservation reserve(Location customer, Map<String, Integer> items) {
        for (Map.Entry<String, Integer> e : items.entrySet()) {
            product(e.getKey());
            if (e.getValue() < 1) throw new IllegalArgumentException("Quantity must be at least 1");
        }
        for (Warehouse w : strategy.rank(customer, warehouses.values())) {
            synchronized (w) {                                  // check and reserve: one step
                boolean hasAll = items.entrySet().stream()
                        .allMatch(e -> w.available(e.getKey()) >= e.getValue());
                if (!hasAll) continue;
                for (Map.Entry<String, Integer> e : items.entrySet()) {
                    Stock s = w.of(e.getKey());
                    int before = s.available();
                    s.reserved += e.getValue();
                    int limit = products.get(e.getKey()).alertBelow();
                    // Alert only when it crosses the line, not on every sale below it
                    if (before >= limit && s.available() < limit)
                        listeners.forEach(l -> l.lowStock(w, e.getKey(), s.available()));
                }
                Reservation r = new Reservation(count.incrementAndGet(), w, items,
                        now.plusMinutes(HOLD_MINUTES));
                reservations.put(r.id, r);
                return r;
            }
        }
        throw new IllegalStateException("No warehouse has all the items");
    }

    Reservation confirm(String id) {
        Reservation r = reservation(id);
        synchronized (r.warehouse) {
```

```
            move(r, Status.CONFIRMED, "confirm");
            r.items.forEach((sku, q) -> {
                Stock s = r.warehouse.of(sku);
                s.onHand -= q;
                s.reserved -= q;
            });
        }
        return r;
    }

    Reservation cancel(String id) {
        Reservation r = reservation(id);
        synchronized (r.warehouse) {
            move(r, Status.CANCELLED, "cancel");
            release(r);
        }
        return r;
    }

    // Moves the clock; returns the holds that ran out, soonest first.
    List<Reservation> setTime(LocalTime t) {
        now = t;
        List<Reservation> expired = new ArrayList<>();
        for (Reservation r : reservations.values()) {
            synchronized (r.warehouse) {
                if (r.status == Status.RESERVED && !t.isBefore(r.expiresAt)) {
                    r.status = Status.EXPIRED;
                    release(r);
                    expired.add(r);
                }
            }
        }
        expired.sort(Comparator.comparing((Reservation r) -> r.expiresAt)
                .thenComparingInt(r -> r.number));
        return expired;
    }

    List<String> report(String sku) {
        product(sku);
        List<String> out = new ArrayList<>();
        for (Warehouse w : new TreeMap<>(warehouses).values()) {          // by name
            synchronized (w) {
                Stock s = w.stock.get(sku);
                if (s != null)
                    out.add(sku + " at " + w.name + ": " + s.onHand + " on hand, " + s.reserved
                            + " reserved, " + s.available() + " available");
            }
        }
        return out;
    }

    private void move(Reservation r, Status next, String verb) {
        if (r.status != Status.RESERVED)
            throw new IllegalStateException("Cannot " + verb + " " + r.id + ": it is " + r.status);
        r.status = next;
    }

    private void release(Reservation r) {
        r.items.forEach((sku, q) -> r.warehouse.of(sku).reserved -= q);
    }

    private Reservation reservation(String id) {
        Reservation r = reservations.get(id);
        if (r == null) throw new IllegalArgumentException("Unknown reservation " + id);
        return r;
    }

    private Warehouse warehouse(String name) {
        Warehouse w = warehouses.get(name);
        if (w == null) throw new IllegalArgumentException("Unknown warehouse " + name);
        return w;
    }

    private Product product(String sku) {
        Product p = products.get(sku);
        if (p == null) throw new IllegalArgumentException("Unknown product " + sku);
        return p;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        PurchasingAlert alerts = new PurchasingAlert();
        Inventory inventory = new Inventory(new NearestWarehouse(), List.of(alerts));
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "WAREHOUSE" -> {
                        inventory.addWarehouse(p[1], loc(p[2], p[3]));
                        System.out.println("Warehouse " + p[1] + " at " + loc(p[2], p[3]));
                    }
                    case "PRODUCT" -> {
                        inventory.addProduct(p[1], Integer.parseInt(p[2]));
                        System.out.println("Product " + p[1] + ", alert below " + p[2]);
                    }
                    case "RESTOCK" -> {
                        Stock s = inventory.restock(p[1], p[2], Integer.parseInt(p[3]));
```

```
                        System.out.println(p[1] + " " + p[2] + ": " + s.onHand + " on hand, "
                                + s.available() + " available");
                    }
                    case "TIME" -> {
                        List<Reservation> expired = inventory.setTime(LocalTime.parse(p[1]));
                        System.out.println("Time " + p[1]);
                        for (Reservation r : expired)
                            System.out.println(r.id + " expired, stock back at " + r.warehouse.name);
                    }
                    case "RESERVE" -> {
                        Location at = loc(p[1], p[2]);
                        Map<String, Integer> items = new LinkedHashMap<>();
                        for (int i = 3; i < p.length; i++) {
                            String[] kv = p[i].split(":");
                            if (kv.length != 2) throw new IllegalArgumentException("Bad item " + p[i]);
                            items.put(kv[0], Integer.parseInt(kv[1]));
                        }
                        Reservation r = inventory.reserve(at, items);
                        List<String> parts = new ArrayList<>();
                        items.forEach((sku, q) -> parts.add(sku + " x" + q));
                        int km = r.warehouse.location.distanceTo(at);
                        System.out.println(r.id + " at " + r.warehouse.name + " (" + km + " km): "
                                + String.join(", ", parts) + ", hold until " + r.expiresAt);
                    }
                    case "CONFIRM" -> {
                        Reservation r = inventory.confirm(p[1]);
                        System.out.println(r.id + " confirmed, ship from " + r.warehouse.name);
                    }
                    case "CANCEL" -> {
                        Reservation r = inventory.cancel(p[1]);
                        System.out.println(r.id + " cancelled, stock back at " + r.warehouse.name);
                    }
                    case "STOCK" -> inventory.report(p[1]).forEach(System.out::println);
                    // n customers each reserve one unit at the same moment
                    case "RUSH" -> {
                        int n = Integer.parseInt(p[2]);
                        Location at = loc(p[3], p[4]);
                        int got = rush(n, i -> {
                            try {
                                inventory.reserve(at, Map.of(p[1], 1));
                                return true;
                            } catch (IllegalStateException e) {
                                return false;
                            }
                        });
                        System.out.println("Rush: " + got + " reserved, " + (n - got) + " refused");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
            alerts.drain().forEach(System.out::println);
        }
    }

    static Location loc(String x, String y) {
        return new Location(Integer.parseInt(x), Integer.parseInt(y));
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

How to read it: `available(sku)` doesn't create a row, so checking Nagpur for a product it never stocked leaves Nagpur unchanged. RUSH's two alerts always come out in the same order: a customer only reaches Pune after seeing Mumbai empty, and Mumbai's alert fired before it emptied.

Part C — model answers

1. Splitting an order. The strategy returns a plan, a list of (warehouse, items): the nearest takes what it can, the rest goes to the next. The reservation gets one part per warehouse. Reserve the parts one at a time and, if a later part fails, release the earlier ones (undo). If you lock several warehouses at once instead, always lock them in the same order, by name, or two orders can deadlock, just like the wallet transfer in PDF 3.

2. A database and 20 servers. One row per (warehouse, SKU). Reserve with a conditional update: `UPDATE stock SET reserved = reserved + :q WHERE warehouse = :w AND sku = :s AND on_hand - reserved >= :q`. One row changed means success. For several items, do the updates in one transaction and roll back if any changes 0 rows. Expiry is a job: `WHERE status = 'RESERVED' AND expires_at <= now()`.

3. Payment arrives after the hold expired. Never drop a paid order. Try to reserve the items again; if they're there, confirm. If not, refund and tell the customer, or ship from another warehouse. To make it rare, give the hold a little more time than the payment page's timeout.

4. One very hot product. Every reservation fights for one row. Split the stock into buckets (10 rows of 100 units; each request picks one, and tries another when it's empty), or keep the count in Redis with an atomic `DECRBY` and write to the database afterwards, or queue reservations for that product and let one thread take them in order.

5. "Keep holds until they pay." "You'd be right if every customer paid. Many close the app instead. A hold that never ends keeps stock out of sale forever: in a sale, everything would look sold out while nothing ships. Fifteen minutes is a business choice we can tune; the expiry job is the small cost."

Common mistakes

- Taking items off on hand at checkout, with no separate reserved count.
- Checking items in one step and reserving them in another, outside the lock.
- Reserving some items, then finding the last one missing and leaving the rest reserved.
- Alerting on every sale below the level, or never again after the first alert.
- Confirming an expired reservation because the status wasn't checked.

Score yourself (1 point each)

☐	On hand, reserved and available kept apart	☐	The nearest warehouse that has everything
☐	Check and reserve under one warehouse lock	☐	All items or none
☐	Confirm, cancel and expiry check the status	☐	Holds expire on the clock, stock released
☐	One alert per drop below the level	☐	Your output matches exactly, RUSH included
☐	Part C 2: the conditional UPDATE	☐	Part C 5: why holds must expire

Solution 4 — Ride-sharing

Questions a strong candidate asks first

Question	Answer assumed here
Which vehicle types, and how is the fare worked out?	Auto, Mini, Sedan; base + per km, fixed up front
How far do we look for a driver?	5 km from the pickup, nearest first
Can a driver get two offers at once?	No: a driver is held while an offer is open
What happens on a decline? On no driver?	Offer the next nearest; with no one left, the ride ends
When does surge apply?	When searching riders outnumber free drivers nearby
Cancellation fees? Ratings?	Rs 50 after the driver accepts; riders rate drivers once

Part A — model answer

```
RideService ———> * Driver (type, location, AtomicReference<DriverStatus>, ratings)
RideService ———> * Ride (rider, type, pickup, drop, fare, declinedBy, driver, RideStatus)
RideService ———> «interface» SurgePolicy       ◁- - -  DemandSurge             VehicleType: enum (base, perKm)
RideService ———> «interface» MatchingStrategy  ◁- - -  NearestDriver

Ride:    OFFERED ——accept——> ACCEPTED ——start——> STARTED ——complete——> COMPLETED
           ├——no driver left——> NO_DRIVER          OFFERED or ACCEPTED ——cancel——> CANCELLED
Driver:  OFFLINE <——> AVAILABLE ——claimed——> OFFERED ——accept——> ON_TRIP ——complete——> AVAILABLE
```

1. Who owns what. A **driver** owns their status, location and ratings. A **ride** owns its life cycle, its fixed fare and who declined it. A **vehicle type** owns its prices. The two **policies** decide how much (surge) and who first (matching). The **service** ties them together.

2. Two strategies, and prices as data (NF1). `SurgePolicy` answers "how much extra?"; `MatchingStrategy` answers "which drivers, in what order?". A Diwali surge cap or "highest rated first" is a new class. The base and per-km prices are just numbers on the enum. The ride's life cycle is an enum with allowed moves, as in Food Delivery.

3. Claim the driver with compareAndSet (NF2). Riders fight over drivers, so the driver is what must be claimed atomically. Matching returns a sorted list, which may already be stale. For each driver, `status.compareAndSet(AVAILABLE, OFFERED)`: exactly one ride wins a driver, and the losers move on to the next. There's no lock over the city, so riders in Kothrud and Viman Nagar never wait for each other. Each ride's own lock covers its life cycle, for example an accept and a cancel at the same moment. Tested: with "check the status, then set it" instead, RUSH gave 3 drivers to 4 rides in **5 runs out of 60**.

4. The ride remembers who declined. `declinedBy` is a set on the ride, and the matching strategy filters it out. A decline frees the driver first, then offers the ride again, so the decliner can't get it back, but other rides can get them.

In the interview: write this first. (1) `Location`, `VehicleType`, `Driver`, `Ride` with both statuses: 20 minutes. (2) `request` with the fare and `offerNext`. (3) Accept and decline. (4) Start, complete, cancel. (5) Surge, ratings, then RUSH. The domain code here is about 215 lines; if time is short, use a fixed fare and say where `SurgePolicy` would go.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.IntPredicate;

record Location(int x, int y) {
    int distanceTo(Location o) { return Math.abs(x - o.x) + Math.abs(y - o.y); }
    @Override public String toString() { return "(" + x + "," + y + ")"; }
}

enum VehicleType {
    AUTO(25, 10), MINI(40, 14), SEDAN(60, 18);

    final int base, perKm;
    VehicleType(int base, int perKm) { this.base = base; this.perKm = perKm; }
}

enum DriverStatus { OFFLINE, AVAILABLE, OFFERED, ON_TRIP }

class Driver {
    final String name;
    final VehicleType type;
    volatile Location location;
    final AtomicReference<DriverStatus> status = new AtomicReference<>(DriverStatus.AVAILABLE);
    private int ratingSum, ratingCount;

    Driver(String name, VehicleType type, Location location) {
        this.name = name; this.type = type; this.location = location;
    }

    // Of many rides trying at once, exactly one gets this driver
    boolean tryClaim() { return status.compareAndSet(DriverStatus.AVAILABLE, DriverStatus.OFFERED); }

    synchronized String rate(int stars) {
        ratingSum += stars;
        ratingCount++;
        return String.format(Locale.ROOT, "%.1f from %d rating%s",
                (double) ratingSum / ratingCount, ratingCount, ratingCount == 1 ? "" : "s");
```

```
    }
}

enum RideStatus {
    OFFERED, ACCEPTED, STARTED, COMPLETED, CANCELLED, NO_DRIVER;

    boolean canMoveTo(RideStatus next) {
        return switch (this) {
            case OFFERED -> next == ACCEPTED || next == CANCELLED || next == NO_DRIVER;
            case ACCEPTED -> next == STARTED || next == CANCELLED;
            case STARTED -> next == COMPLETED;
            default -> false;
        };
    }
}

// Changed only while holding its own lock: synchronized (ride)
class Ride {
    final String id, rider;
    final VehicleType type;
    final Location pickup, drop;
    final int fare, surgePercent;
    final Set<Driver> declinedBy = new HashSet<>();
    RideStatus status = RideStatus.OFFERED;
    Driver driver;
    boolean rated;

    Ride(String id, String rider, VehicleType type, Location pickup, Location drop, int fare,
         int surgePercent) {
        this.id = id; this.rider = rider; this.type = type; this.pickup = pickup; this.drop = drop;
        this.fare = fare; this.surgePercent = surgePercent;
    }
}

interface SurgePolicy {
    int percent(int searching, int freeDrivers);
}

// 1.5x when riders waiting for a driver outnumber free drivers nearby
class DemandSurge implements SurgePolicy {
    public int percent(int searching, int freeDrivers) { return searching > freeDrivers ? 150 : 100; }
}

interface MatchingStrategy {
    List<Driver> candidates(Ride ride, Collection<Driver> all);    // best first
}

class NearestDriver implements MatchingStrategy {
    static final int RADIUS_KM = 5;

    public List<Driver> candidates(Ride ride, Collection<Driver> all) {
        return all.stream()
                .filter(d -> d.type == ride.type && d.status.get() == DriverStatus.AVAILABLE)
                .filter(d -> d.location.distanceTo(ride.pickup) <= RADIUS_KM)
                .filter(d -> !ride.declinedBy.contains(d))
                .sorted(Comparator.comparingInt((Driver d) -> d.location.distanceTo(ride.pickup))
                        .thenComparing(d -> d.name))
                .toList();
    }
}

class RideService {
    static final int CANCEL_FEE = 50, DRIVER_SHARE_PERCENT = 80;
    private final Map<String, Driver> drivers = new ConcurrentHashMap<>();
    private final Map<String, Ride> rides = new ConcurrentHashMap<>();
    private final Map<String, Ride> activeByRider = new ConcurrentHashMap<>();
    private final AtomicInteger count = new AtomicInteger();
    private final SurgePolicy surge;
    private final MatchingStrategy matching;

    RideService(SurgePolicy surge, MatchingStrategy matching) {
        this.surge = surge; this.matching = matching;
    }

    void addDriver(String name, VehicleType type, Location l) {
        if (drivers.putIfAbsent(name, new Driver(name, type, l)) != null)
            throw new IllegalArgumentException("Driver " + name + " already exists");
    }

    void goOnline(String name) {
        if (!driver(name).status.compareAndSet(DriverStatus.OFFLINE, DriverStatus.AVAILABLE))
            throw new IllegalStateException(name + " is already online");
    }

    void goOffline(String name) {
        Driver d = driver(name);
        if (!d.status.compareAndSet(DriverStatus.AVAILABLE, DriverStatus.OFFLINE))
            throw new IllegalStateException(name + " is " + d.status.get() + ", cannot go offline");
    }

    Ride request(String rider, VehicleType type, Location pickup, Location drop) {
        Ride current = activeByRider.get(rider);
        if (current != null)
            throw new IllegalStateException(rider + " already has a ride (" + current.id + ")");
        long searching = 1 + rides.values().stream()
                .filter(r -> r.type == type && r.status == RideStatus.OFFERED).count();
        long free = drivers.values().stream()
                .filter(d -> d.type == type && d.status.get() == DriverStatus.AVAILABLE)
                .filter(d -> d.location.distanceTo(pickup) <= NearestDriver.RADIUS_KM).count();
        int percent = surge.percent((int) searching, (int) free);
        int fare = (type.base + type.perKm * pickup.distanceTo(drop)) * percent / 100;
```

```
        Ride ride = new Ride("R" + count.incrementAndGet(), rider, type, pickup, drop, fare, percent);
        rides.put(ride.id, ride);
        activeByRider.put(rider, ride);
        synchronized (ride) {
            offerNext(ride);
        }
        return ride;
    }

    Ride accept(String id) {
        Ride ride = ride(id);
        synchronized (ride) {
            move(ride, RideStatus.ACCEPTED, "accept");
            ride.driver.status.set(DriverStatus.ON_TRIP);
        }
        return ride;
    }

    // Returns the driver who declined; the ride is offered to the next one, if any.
    Driver decline(String id) {
        Ride ride = ride(id);
        synchronized (ride) {
            if (ride.status != RideStatus.OFFERED)
                throw new IllegalStateException("Cannot decline " + id + ": it is " + ride.status);
            Driver d = ride.driver;
            ride.declinedBy.add(d);
            d.status.set(DriverStatus.AVAILABLE);
            offerNext(ride);
            return d;
        }
    }

    Ride start(String id) {
        Ride ride = ride(id);
        synchronized (ride) {
            move(ride, RideStatus.STARTED, "start");
        }
        return ride;
    }

    Ride complete(String id) {
        Ride ride = ride(id);
        synchronized (ride) {
            move(ride, RideStatus.COMPLETED, "complete");
            ride.driver.location = ride.drop;
            free(ride);
        }
        return ride;
    }

    // Returns the cancellation fee.
    int cancel(String id) {
        Ride ride = ride(id);
        synchronized (ride) {
            int fee = ride.status == RideStatus.ACCEPTED ? CANCEL_FEE : 0;
            move(ride, RideStatus.CANCELLED, "cancel");
            free(ride);
            return fee;
        }
    }

    String rate(String id, int stars) {
        if (stars < 1 || stars > 5) throw new IllegalArgumentException("Stars must be 1 to 5");
        Ride ride = ride(id);
        synchronized (ride) {
            if (ride.status != RideStatus.COMPLETED)
                throw new IllegalStateException("Cannot rate " + id + ": it is " + ride.status);
            if (ride.rated) throw new IllegalStateException(id + " is already rated");
            ride.rated = true;
            return ride.driver.rate(stars);
        }
    }

    // Caller holds the ride's lock
    private void offerNext(Ride ride) {
        for (Driver d : matching.candidates(ride, drivers.values())) {
            if (d.tryClaim()) {                                  // someone faster? try the next
                ride.driver = d;
                return;
            }
        }
        ride.driver = null;
        move(ride, RideStatus.NO_DRIVER, "match");
        activeByRider.remove(ride.rider);
    }

    private void free(Ride ride) {
        if (ride.driver != null) ride.driver.status.set(DriverStatus.AVAILABLE);
        activeByRider.remove(ride.rider);
    }

    private void move(Ride ride, RideStatus next, String verb) {
        if (!ride.status.canMoveTo(next))
            throw new IllegalStateException("Cannot " + verb + " " + ride.id + ": it is "
                    + ride.status);
        ride.status = next;
    }

    Ride ride(String id) {
        Ride r = rides.get(id);
        if (r == null) throw new IllegalArgumentException("Unknown ride " + id);
```

```
        return r;
    }

    private Driver driver(String name) {
        Driver d = drivers.get(name);
        if (d == null) throw new IllegalArgumentException("Unknown driver " + name);
        return d;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        RideService service = new RideService(new DemandSurge(), new NearestDriver());
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "DRIVER" -> {
                        service.addDriver(p[1], type(p[2]), loc(p[3], p[4]));
                        System.out.println("Driver " + p[1] + " (" + p[2] + ") at " + loc(p[3], p[4]));
                    }
                    case "ONLINE" -> {
                        service.goOnline(p[1]);
                        System.out.println(p[1] + " is online");
                    }
                    case "OFFLINE" -> {
                        service.goOffline(p[1]);
                        System.out.println(p[1] + " is offline");
                    }
                    case "REQUEST" -> {
                        Ride r = service.request(p[1], type(p[2]), loc(p[3], p[4]), loc(p[5], p[6]));
                        if (r.driver == null) {
                            System.out.println(r.id + ": no " + r.type + " near " + r.pickup);
                        } else {
                            String surge = r.surgePercent > 100 ? " (surge 1.5x)" : "";
                            System.out.println(r.id + ": " + r.rider + ", " + r.type + ", "
                                    + r.pickup.distanceTo(r.drop) + " km, Rs " + r.fare + surge
                                    + ", offered to " + offer(r));
                        }
                    }
                    case "ACCEPT" -> {
                        Ride r = service.accept(p[1]);
                        System.out.println(r.driver.name + " accepted " + r.id);
                    }
                    case "DECLINE" -> {
                        Driver d = service.decline(p[1]);
                        Ride r = service.ride(p[1]);
                        System.out.println(d.name + " declined " + r.id + ", "
                                + (r.driver == null ? "no other driver" : "offered to " + offer(r)));
                    }
                    case "START" -> System.out.println(service.start(p[1]).id + " started");
                    case "COMPLETE" -> {
                        Ride r = service.complete(p[1]);
                        int share = r.fare * RideService.DRIVER_SHARE_PERCENT / 100;
                        System.out.println(r.id + " completed: " + r.rider + " pays Rs " + r.fare + ", "
                                + r.driver.name + " gets Rs " + share);
                    }
                    case "CANCEL" -> {
                        int fee = service.cancel(p[1]);
                        System.out.println(p[1] + " cancelled, "
                                + (fee == 0 ? "no fee" : "fee Rs " + fee));
                    }
                    case "RATE" -> {
                        String summary = service.rate(p[1], Integer.parseInt(p[2]));
                        System.out.println(service.ride(p[1]).driver.name + " rated " + p[2] + ", now "
                                + summary);
                    }
                    // n riders at the same pickup request at the same moment
                    case "RUSH" -> {
                        int n = Integer.parseInt(p[6]);
                        VehicleType t = type(p[1]);
                        int got = rush(n, i -> service.request("Rider" + i, t, loc(p[2], p[3]),
                                loc(p[4], p[5])).driver != null);
                        System.out.println("Rush: " + got + " got a driver, " + (n - got)
                                + " found none");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    static String offer(Ride r) {
        return r.driver.name + " (" + r.driver.location.distanceTo(r.pickup) + " km away)";
    }

    static Location loc(String x, String y) {
        return new Location(Integer.parseInt(x), Integer.parseInt(y));
    }

    static VehicleType type(String s) {
        try {
            return VehicleType.valueOf(s);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Unknown vehicle type " + s);
        }
```

```
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

How to read it: surge is worked out before the driver is claimed, because the rider sees the fare before anyone accepts. A ride's lock can be held while claiming a driver, but claiming isn't a lock, so two locks are never taken in opposite orders and nothing can deadlock.

Part C — model answers

1. A 15-second timeout. When an offer is made, schedule a check for 15 seconds later (a `ScheduledExecutorService`). When it runs, inside the ride's lock: if the ride is still OFFERED to the same driver, decline for them. If the driver accepted in time, the status check makes the timeout do nothing. It's the same idea as BookMyShow's hold expiry.

2. A million moving drivers. Split the city into cells (geohash, or Uber's H3 hexagons) and keep the free drivers of each cell in memory or Redis (`GEOADD`, `GEOSEARCH`). To find the nearest, look in the pickup's cell and its neighbours and rank those few drivers. A location update just moves a driver between cells. Split the whole system by city.

3. Shared rides. A trip gets seats and several riders with their own pickups and drops. Matching also considers drivers already on a shared trip with a free seat and a small detour. That's mostly a new `MatchingStrategy` and a trip with several stops; each rider's ride keeps its own life cycle and fare.

4. The driver's app crashes. The server owns the trip, not the phone. When the app comes back, it reloads the trip from the server. If the driver's heartbeats stop for several minutes, alert support; the fare can come from the planned route and the last known location. A crash must never complete or cancel a trip by itself.

5. "Just an if for surge." "You'd be right that today it's one rule: 1.5× when riders outnumber drivers. But surge changes often: different cities, times of day, caps during festivals, a regulator limiting it to 2×. With an interface, each change is a new class tested on its own, and the fare code stays the same. Today the interface costs three lines."

Common mistakes

- Checking that a driver is free and marking them busy in two separate steps.
- One lock around all matching, so every request in the city waits in one line.
- Offering a ride again to the driver who just declined it.
- Forgetting to free the driver on cancel, or to move them to the drop point on completion.
- Working out the fare again at the end, so the rider pays a different amount than shown.

Score yourself (1 point each)

☐	Ride and driver statuses drawn separately	☐	The fare is fixed at request time
☐	Surge and matching are strategies	☐	Nearest within 5 km, ties by name
☐	A driver is claimed with compareAndSet	☐	Decline offers the next, never the same driver
☐	Cancel fee only after acceptance	☐	Your output matches exactly, RUSH included
☐	Part A 3: why no global lock	☐	Part C 2: cells for nearby search

Back pages

Warm-up answers

W1 — Task Scheduler. A `TreeSet` holds tasks not yet due, by due time (then creation); a `PriorityQueue` holds due tasks waiting for a worker, by priority, then due time, then creation. Each second: (1) finish runs ending now, (2) move everything due into the waiting queue, (3) give idle workers the head of the queue. Fixed delay (next due = finish + interval), so a slow run never overlaps the next. Cancel removes the task from both structures; a running run finishes, and nothing more is scheduled.

W2 — Food Delivery. PLACED → ACCEPTED or REJECTED → PICKED_UP → DELIVERED, with CANCELLED from PLACED or ACCEPTED. Every move does the same three things (check, set, notify), so an enum with `canMoveTo` is enough; State earns its place when states behave differently. Choosing a free partner and marking them busy happen under one lock. A freed partner (after delivery or cancel) immediately takes the oldest waiting order.

#	W3 answer	Why
1	**Strategy**	charging rules that keep changing.
2	**Nothing**: an enum with allowed moves	every step does the same thing; only the allowed moves differ.
3	**Adapter**	an old interface we can't change, behind our own.
4	**Observer**	reactions to one event, with more to come.
5	**Chain of Responsibility**	up to X this one decides; otherwise pass it on.
6	**Decorator**	extras in any combination, each wrapping the last.
7	**Composite**	a whole and its parts answer the same question.
8	**Nothing**: a scheduled job	"2 days have passed" isn't an event.
9	**Template Method**	fixed steps in a fixed order; two of them vary.
10	**Factory**: a map from name to supplier	pick an implementation by a name in the config.
11	**Proxy**	the same interface, with an access check and a cache in front.
12	**Nothing**: a record	three fields that never change.

Both designs right and 10 or more of the 12: on track. If W1 or W2 was shaky, re-do that problem's Part A from PDF 5 before Problem 2 here.

Mistake log

One row per mistake, in your own words. Before your first mock interview, read this page and every earlier log once.

Date	Problem	What I did	What's right	Rule to remember

Re-do schedule

Problem	Attempted (time taken)	Day 3: Part A from memory (15 min)	Day 10: Part A + the core class	Day 30: the whole problem, timed	Score /10
Amazon Locker					
Notification System					
Inventory Management					
Ride-sharing					

The core class to rebuild on day 10: `Locker` with `findFree` and `pickUp` · `NotificationService.send` and `deliver` · `Inventory.reserve` and `setTime` · `RideService`'s `request`, `decline` and `offerNext`. PDF 5's day-30 re-dos are probably due around now too.

All 24 problems on one page

If you remember one idea per problem, it should be this one. Before a mock interview, cover the right column and say each idea out loud.

PDF	Problem	The idea to remember
1	Tic-Tac-Toe	Counters per row, column and diagonal give a win check in constant time; Board and Game are separate.
1	Vending Machine	State pattern: each state answers coin, select and refund differently. Check you can give change before taking money.
1	Library Management	A hold queue per book: a returned copy goes to the next hold first. Member types are an enum of numbers, not classes.
1	Parking Lot	Finding and taking a spot is one indivisible step. Spot choice and fees are policies; displays are observers.
2	Teen Patti hands	Hand ranking is a Comparator. Pass the randomness in, so shuffling can be tested.
2	ATM	Note cassettes as a Chain of Responsibility; plan the notes before giving any. The wrong-PIN count lives on the card.
2	Splitwise	One net balance per person; split types are strategies; amounts in paise, never `double`.
2	Elevator System	"Which lift?" and "where does a lift go next?" are separate decisions. Each lift has its own lock.
3	LRU Cache	A HashMap plus a doubly linked list: every operation in O(1).
3	Rate Limiter	The algorithm is a strategy, and each limiter is thread-safe on its own.
3	Digital Wallet	Lock both wallets, always in the same order, or two transfers can deadlock.
3	BookMyShow	Seat holds that expire; one step per show; the booking status is an enum, not State.
4	Snake and Ladder	One map of jumps instead of Snake and Ladder classes; a Dice interface for tests.
4	Cache with TTL	Lazy plus active expiry; the clock is passed in; two orders need two structures.
4	Meeting Room Scheduler	Clash when s1 < e2 and s2 < e1; one `lowerEntry` finds it; one lock per room.
4	Chess	"Leaves my king in check": try the move and undo it. Mate and stalemate ask the same question.
5	Logging Framework	The effective level walks up the dots to root; where a line goes and how it looks vary separately.
5	Hotel Booking	A stay is the nights check-in to check-out − 1; the lowest free room of the type, under that room's lock.
5	Task Scheduler	A timeline of future tasks plus a priority queue of due ones; each second: finish, move due, assign.
5	Food Delivery	An enum life cycle; choose a partner and mark them busy in one step; a freed partner takes the oldest waiting order.
6	Amazon Locker	The smallest compartment that fits, by enum order; codes come from a generator that's passed in; pickup is a lookup.
6	Notification System	Validate first, then one atomic `add` stops duplicates; retries per channel, failures kept apart.
6	Inventory Management	On hand, reserved and available; all items or none under the warehouse's lock; alert when the line is crossed.
6	Ride-sharing	Claim a driver with `compareAndSet`, with no lock over the city; surge and matching are strategies.

The patterns behind all 24. Strategy wherever a rule will change. An enum with allowed moves for most life cycles; State only when states behave differently. Observer for reactions. Pass in anything random or time-based so tests are repeatable. For concurrency, find the one check-then-act step, lock the smallest thing that covers it, and test with a rush.

That's Phase 5 done. Next is **Phase 6, mock interviews**. Once a week, take a problem you haven't touched for three weeks and do it in 90 minutes, talking out loud the whole time. If you can, have a friend read you the purple box, answer your questions from the requirements, and ask the Part C questions. Then score yourself with the checklist. Keep doing the re-dos on schedule: they're what makes the patterns automatic.